



UNIVERSITY OF
PLYMOUTH

School of Engineering,
Computing & Mathematics

Session 6: Front-end Frameworks (jQuery, Angular & React)

Full-Stack Development

Mark Dixon

School of Engineering, Computing and Mathematics

Scrum Standup: Last Session

- What did we do last session
 - Progress:
did you learn anything (was anything particularly useful or good)?
(write down topics, what you can remember)
 - Problems / Blockers:

How did the lab session go (were you able to get something running)?

What could be better?

Introduction

Today's topics

1. JQuery
2. Angular
3. React.js

Session learning outcomes – **by the end of today's lecture you will be able to:**

- Use JQuery in your front-end application(s)
- Create simple Angular apps using Angular CLI
- Create single-page apps with the Angular router
- Create simple React.js apps

jQuery: how to include it

Download and include

```
<head>  
  <script src="jquery-3.3.1.min.js"></script>  
</head>
```

Reference online version

```
<head>  
  <script src="https://ajax.googleapis.com/ajax/  
    libs/jquery/3.3.1/jquery.min.js"></script>  
</head>
```

Trade-offs

- Online – Cached
- Online – Will not work without Offline
- Online – Faster? (Loaded from closest server)

jQuery: “write less, do more”

- Cross-browser
- Free course: <http://try.jquery.com>
- w3schools tutorial: <http://www.w3schools.com/jquery>

```
function changeWidth(newWidth) {  
    $("#mydiv").width(newWidth);  
}  
  
$(function() {  
    $("#mydiv").click(function() {  
        let newWidth = prompt("Enter new width");  
        changeWidth(newWidth);  
    });  
});
```

← window onLoad

jQuery: syntax

Selecting DOM elements

```
let textDiv = document.getElementById("textDiv");           // JS
let listDiv = document.getElementsByClassName("listDiv");

let textDiv = $("#textDiv");                                 // Jqry
let listDiv = $(".listDiv");
```

Ready?

```
window.onload = function() {                                // JS
    document.getElementById("btn1").addEventListener("click", setTextColour);
    document.getElementById("btn2").addEventListener("click", setBgColour);
}

$(function() {                                              // Jqry
    $("#btn1").click(setTextColour);
    $("#btn2").click(setBgColour);
});
```

jQuery Ajax...

get function

- **\$.ajax()** – jQuery's base Ajax function, very powerful
- **get** function calls base function
- **wrapper** functions (such as get) usually sufficient
- **\$.get(...)** – arguments are **URL** & **event handler**

```
$(function() {  
    $.get("http://localhost/rand", function(data) {  
        $("#randnum").html(data);  
    });  
});
```

Full-Stack JavaScript

MongoDB

- NoSQL database

Express.js

- Resource mapping for RESTful web APIs

Angular / React

- Dynamic data binding for web MVC

Node.js



JavaScript approaches to object orientation

Implement OO with JS features yourself

- Use prototypes and instances as classes and objects

Using a third party library/language

- **TypeScript** – supports full OO features and syntax
 - types, class, extends, interface

Rethink OO

- Question the value of individual OO concepts

TypeScript

- TypeScript = JavaScript + static typing
- TypeScript – superset of JavaScript
- Therefore, all JavaScript – syntactically correct TypeScript

TypeScript

```
type Result = "pass" | "fail"

function verify(res: Result) {
  if (res === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

Add Types

Remove Types

JavaScript

```
function verify(result) {
  if (result === "pass") {
    console.log("Passed")
  } else {
    console.log("Failed")
  }
}
```

Angular

A client-side framework

Single-page applications

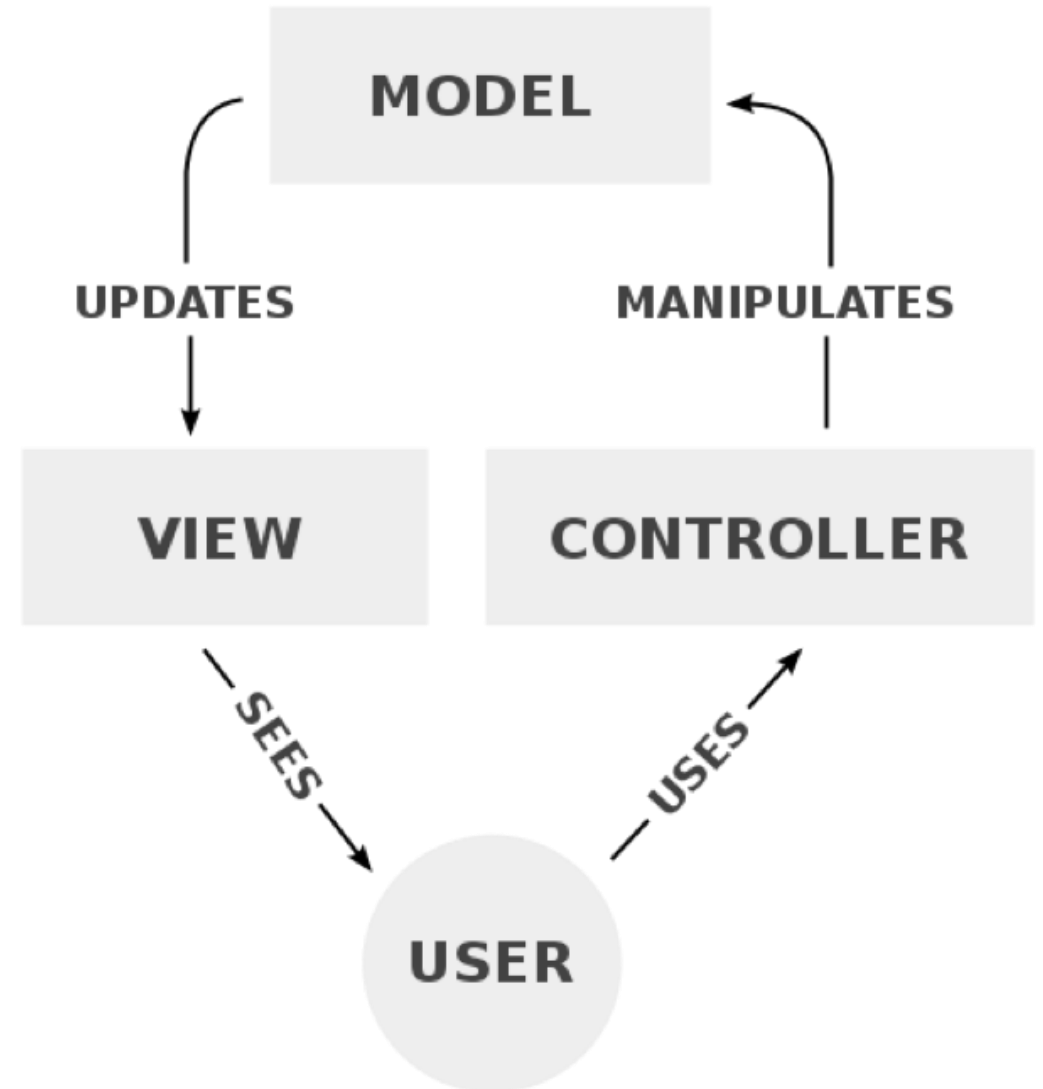
MVC (model-view-controller) capabilities

- Model (scope) – variables
- View (DOM)
- Controller

Extends HTML

- Dynamic two-way data binding

Built on TypeScript (*.ts)



Angular: Hello World

Install Angular CLI with (or add to package.json):

```
npm install -g @angular/cli@18.2.21
```

Use **ng new** command within Angular CLI to create project

- Much of the code is provided for you
- Adapt the template code to suit your application needs

```
ng new --standalone=false my-project-name
```

Hello World app

- Modify **app.component.html** to construct the view

```
<p>{{message}}</p>
```

- Modify **app.component.ts** to set up the model...

LECTURE-SLIDES

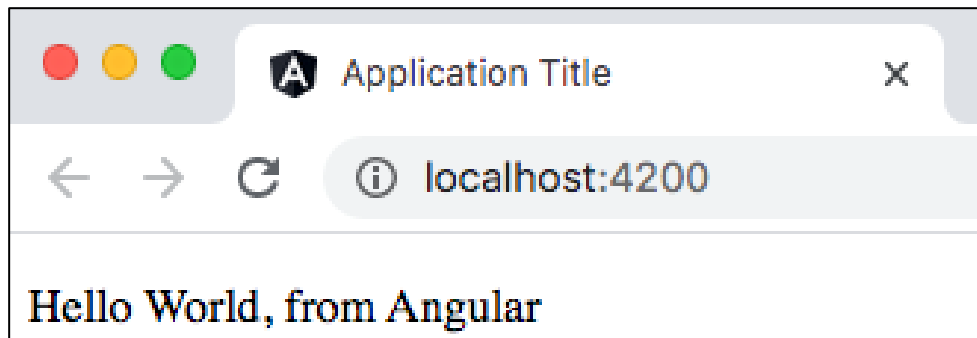
- > e2e
- > node_modules
- ▼ src
 - ▼ app
 - TS app-routing.module.ts
 - # app.component.css
 - <> app.component.html
 - TS app.component.spec.ts
 - TS app.component.ts
 - TS app.module.ts
 - > assets
 - > environments
 - ★ favicon.ico
 - <> index.html
 - TS main.ts
 - TS polyfills.ts
 - # styles.css
 - TS test.ts
 - ≡ .browserslistrc
 - ⚙ .editorconfig
 - 💎 .gitignore
 - { } angular.json
 - 🔧 karma.conf.js
 - { } package-lock.json
 - { } package.json
 - 📖 README.md
 - { } tsconfig.app.json
 - { } tsconfig.json
 - { } tsconfig.spec.json
 - { } tslint.json

Angular: Hello World

```
import { Component } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
  styleUrls: ['./app.component.css']
})

export class AppComponent {
  title = 'lecture-slides';
  message = "Hello World, from Angular";
}
```



This is the only change to the generated code

Angular: Project Size

- Angular imports a large number of NPM modules
- Makes project folder large
(most of this is in node_modules folder)

Size: 276 MB (290,081,978 bytes)

Size on disk: 317 MB (333,238,272 bytes)

Contains: 41,124 Files, 3,691 Folders

Angular: Model

Define (or obtain) the data in the model

```
export class AppComponent {  
    title = 'lecture-slides';  
  
    students = [  
        {name: "Abdoulaye Willey", course: "Computer Science"},  
        {name: "Lisette Dennis", course: "Computing"},  
        {name: "Chen Yating Liao", course: "Computng & Software Development"},  
        {name: "Marie Kendall", course: "Computing & Games Development"},  
        {name: "Brandon Weeber", course: "Games Development Technologies"},  
    ]  
}
```

Angular: View

Reference from within the view

- Use ***ngFor** to loop over the array of students
- Embed each student within a **table row** element

```
<table>
  <tr><th>Student</th><th>Course</th></tr>
  <tr *ngFor="let student of students">
    <td>{{ student.name }}</td>
    <td>{{ student.course }}</td>
  </tr>
</table>
```

Student	Course
Abdoulaye Willey	Computer Science
Lisette Dennis	Computing
Chen Yating Liao	Computing & SD
Marie Kendall	CGD
Brandon Weeber	GDT

Angular: Filtering data

```
<table>
  <tr><th>Student</th><th>Course</th></tr>
  <ng-container *ngFor="let student of students">
    <tr *ngIf="student.course == 'Computer Science'">
      <td>{{ student.name }}</td>
      <td>{{ student.course }}</td>
    </tr>
  </ng-container>
</table>
```

Student	Course
Abdoulaye Willey	Computer Science

Angular: Dynamically filtering data

app.component.html

```
<table>
  <tr><th>Student</th><th>Course</th></tr>
  <ng-container *ngFor="let student of students">
    <tr *ngIf="student.course == filter">
      <td>{{ student.name }}</td>
      <td>{{ student.course }}</td>
    </tr>
  </ng-container>
</table>
<input [(ngModel)]="filter" />
```

app.component.ts

```
export class AppComponent {
  filter: string = '';
}
```

Student	Course
Lisette Dennis	Computing
Computing	

Angular: Ajax

1. Modify **app.module.ts** to include the **HttpClientModule**
2. Include **ngOnInit** method in which the Ajax call is made

app.module.ts

- Add **HttpClientModule** to a list of imports in **app.module.ts**

```
import { HttpClientModule } from '@angular/common/http';
```

```
imports: [  
    BrowserModule,  
    FormsModule,  
    AppRoutingModule,  
    HttpClientModule  
],
```

Angular: Ajax

app.component.ts

- Declare an object to store our data and pass to the view
- Dependency injection – inject the **HttpClient** interface
- Declare a **ngOnInit** method that makes the Ajax call and puts the result into the **students** object

```
export class AppComponent {  
  students : Object;  
  
  constructor(public httpClient: HttpClient) {}  
  
  ngOnInit() {  
    this.httpClient.get("http://localhost:9000/")  
      .subscribe((res) => {  
        this.students = res["students"];  
      });  
  }  
}
```

Angular: Components

- Divide the application into components – **vertically slice** to combine view and controller
- Promotes re-usability
- A component **myComponent** can be included in a template with **<my-component></my-component>**
- Generate using Angular CLI with **ng generate new template my-component**
- For a component called **Module:**

```
<app-module code="COMP3006" title="Full-Stack Development"></app-module>  
<app-module code="COMP2002" title="Artificial Intelligence"></app-module>
```

COMP3006: *Full-Stack Development*

COMP2002: *Artificial Intelligence*

Angular: Components

```
import { Component, Input, OnInit } from '@angular/core';

@Component({
  selector: 'app-module',
  templateUrl: './module.component.html',
  styleUrls: ['./module.component.css']
})

export class ModuleComponent implements OnInit {

  @Input() code : string = "";
  @Input() title : string = "";

  constructor() {}

  ngOnInit(): void {}

}
```

Angular: Routing

- Works in the same way as in **Express**
- Route between **views**

```
import { NgModule } from '@angular/core';
import { Routes, RouterModule } from '@angular/router';
import { CompOneComponent } from '../comp-one/comp-one.component';
import { CompTwoComponent } from '../comp-two/comp-two.component';

const routes: Routes = [
  {path: "one", component: CompOneComponent},
  {path: "two", component: CompTwoComponent},
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})

Export class AppRoutingModule {}
```

Angular: Routing

```
ng generate component comp-one  
ng generate component comp-two
```

comp-one.component.html

```
<p>Component One</p>
```

comp-one.component.css

```
p { color: red; font-size: 24px; }
```

comp-two.component.html

```
<p>Component Two</p>
```

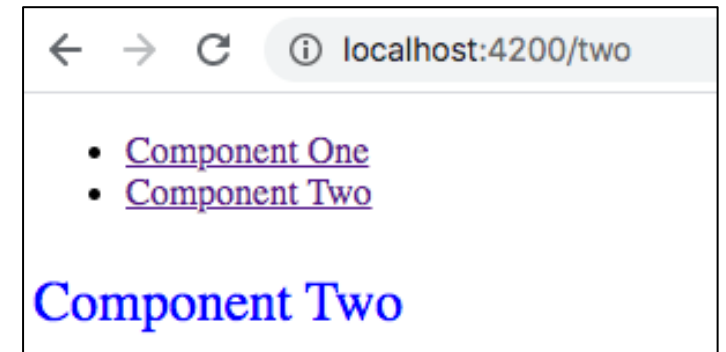
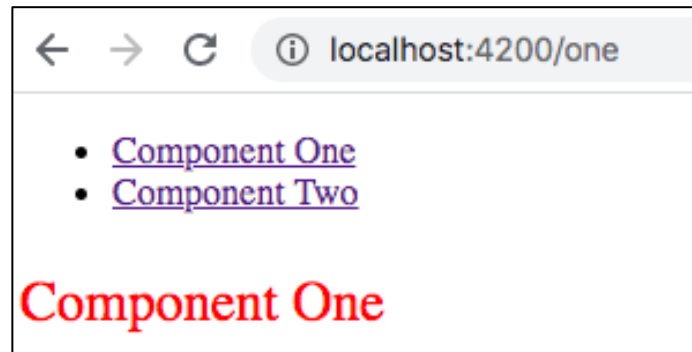
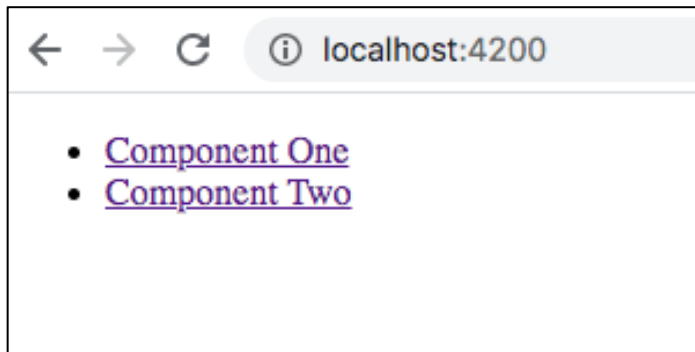
comp-two.component.css

```
p { color: red; font-size: 24px; }
```


Angular: Routing

```
<ul>
  <li><a href="/one">Component One</a></li>
  <li><a href="/two">Component Two</a></li>
</ul>

<router-outlet></router-outlet>
```



React.js

- Free
- Open-source
- Front-end JavaScript library

- Single page applications
- Mobile apps
- Server-rendered apps

- Declarative (vs Imperative)
- Hot reloading

React Hello World

- Create React App (CRA) tool
 - Single terminal command

```
npx create-react-app empty-react
```

- Takes a while (similar to Angular)
- Generates quite a large project

Size: 256 MB (268,439,659 bytes)

Size on disk: 299 MB (314,478,592 bytes)

Contains: 39,382 Files, 5,038 Folders

- To run:

```
npm start
```

React Component - Hello World

- Create HelloWorld.js file (inside src folder):

```
import React from 'react';

const HelloWorld = () => {

  function sayHello() {
    alert('Hello there!!!');
  }

  return (
    <button onClick={sayHello}>Click me!</button>
  )
};

export default HelloWorld;
```

- Add the following to App.js (inside the className div tags):

```
<HelloWorld />
```

Exercise 1: Angular application

- You can do this inside a Docker container (using the same approach we used with Express.js last week), or directly on your host machine.
- Install Angular CLI with the command:

```
npm install -g @angular/cli@18.2.21
```

- Then create a sample application – called **hello-app**:

```
ng new --standalone=false hello-world
```

- Change directory into the top-level directory of the app:

```
cd hello-world
```

- Finally, run the app and confirm that you can see it in your browser (your browser should open automatically):

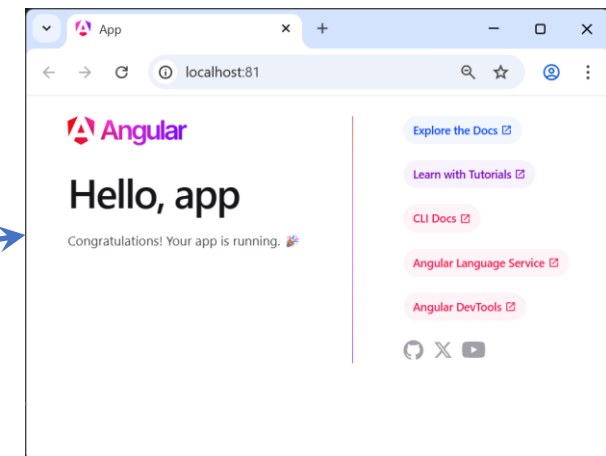
```
ng serve --open
```

- You should see something like this
- Or inside a Docker container:

```
ng serve --host=0.0.0.0 --disable-host-check
```

This might take a few minutes.

When presented with options say **no** and choose the **CSS** option for styling.



Exercise 2: Angular application

- You will now construct the app, for which you will need to do three things. The good news is most of the code has been generated for you by Angular, so you just have to modify the files as follows.
- **app.module.ts:** The app will bind a form input field to a model attribute, so you need to add **FormsModule** to the array of imports, which will look like this when you've done it:

```
imports: [  
  BrowserModule,  
  FormsModule,  
],
```

- Once you've added this you need to make sure you've imported **FormsModule** (add the following to the top of the file, if your IDE hasn't already done it):

```
import { FormsModule } from '@angular/forms';
```

- **app.component.ts:** The next step is to modify the component file. Modify the **AppComponent** class by adding a name attribute as follows:

```
export class AppComponent {  
  title = 'hello-world';  
  name = "";  
}
```

Exercise 2: Angular application

- **app.component.html:** Delete the code in the template file and replace it with the following:

```
<h1>Hello World app</h1>
<p>
  Enter your name:
  <input [(ngModel)]="name" />
</p>
<p>Your name is {{name}}</p>
```

- If your app is still running it should refresh your changes. If not, run the app again and confirm that you see something like this in your browser:



Hello World app

Enter your name:

Your name is David

- When you type your name into the box you should see it appear in the paragraph.

Exercise 3: Angular components

In this exercise you will create an app that uses an additional component. The aim is to use Angular to re-create the train information board you created in a previous session. You may find it helpful to refresh yourself with that exercise.

- Create a new app called train-app with the ng new command as in Exercise 1, and run the application.
- app.module.ts: Add HttpClientModule to the imports (and import HttpClientModule) from angular/common/http if your IDE hasn't done it for you). This will enable us to make an Ajax call to the server:(<https://web.socem.plymouth.ac.uk/COMP3006/trains/trains>).
- app.component.ts: The AppComponent class initiates the app by making an Ajax call to the API and storing the result in an attribute called trains. Modify the AppComponent class so that it appears as follows:

```
export class AppComponent {  
  title = 'train-app';  
  
  trains: any[] = [];  
  
  constructor(public httpClient: HttpClient) {}  
  
  ngOnInit() {  
    this.httpClient.get("https://web.socem.plymouth.ac.uk/COMP3006/trains/trains /")  
      .subscribe((res: any) => {  
        this.trains = res.trains;  
      });  
  }  
}
```


Exercise 3: Angular components

- **app.component.html:** We'll use the ***ngFor** directive to loop over the trains and one-by-one add an **app-train** element to the page. The app (the parent component) will pass three pieces of information to the child component (which we'll create shortly): the destination, the departure time, and the platform. Remove the existing template and add the following:

```
<p class="heading departs">Time</p>
<p class="heading destination">Destination</p>
<p class="heading platform">Platform</p>

<div *ngFor="let train of trains ">
  <app-train
    [destination]="train.stops[train.stops.length-1].name"
    [departs]="train.departs"
    [platform]="train.platform"></app-train>
</div>
```

- Create a new component with the following command:

```
ng generate component train
```

- **train.component.ts:** Modify the start of the TrainComponent class as follows:

```
export class TrainComponent implements OnInit {

  @Input() departs: string = '';
  @Input() destination: string = '';
  @Input() platform = '';

  constructor() { }
```

Exercise 3: Angular components

- **train.component.html:** Add the following to the template in order to present the information about each specific train:

```
<p class="departs">{{departs}}</p>
<p class="destination">{{destination}}</p>
<p class="platform">{{platform}}</p>
```

- Finally, apply the styling to the app. Some is shared between the components (this goes into the styles.css file) while some is bespoke to one of the two components (app.component.css and train.component.css)
- **styles.css:** Add the following to the stylesheet:

```
p { font-family: Arial, Helvetica, sans-serif; font-size: 24px; }
p.departs, p.destination, p.platform { float: left; margin: 0; }
.departs, .platform { width: 20%; }
.destination { width: 60%; }
.platform { clear: right; }
```

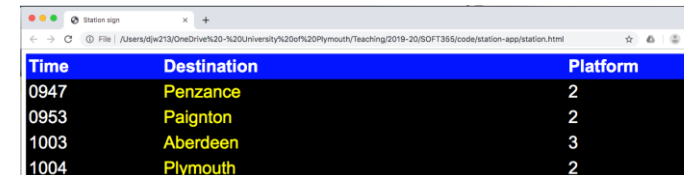
- **app.component.css:** Add the following to the stylesheet:

```
p.heading { color: #FFF; background-color: #0000FF; }
```

- **train.component.css:** Add the following to the stylesheet:

```
p.departs, p.destination, p.platform { background-color: #000; }
.destination { color: #FFFF00; }
.departs, .platform { color: #FFF; }
```

- When you've finished, you should have something like this in your browser:



A screenshot of a web browser window displaying a table with train departure information. The table has three columns: Time, Destination, and Platform. The data is as follows:

Time	Destination	Platform
0947	Penzance	2
0953	Paignton	2
1003	Aberdeen	3
1004	Plymouth	2